Joshua Gould - 1995460
Kekoa Sylva - 2495199

PARK PICNIC PLANNER

Final Report

Project Aim & Description

The mobile Park Picnic Planner app is designed for the purpose of giving users an easy-to-use way to coordinate outings in public parks, and provide realtime communication and updates for those events. The app allows users to personalize their profile, join and/or create events, and even view open-invite events from other users or groups.

The app was built using Flutter/dart and the Material UI library. The database utilized was Firebase, in particular, Firestore, a NoSQL document object model database that is part of Google's web services offerings. No dedicated back-end was created to facilitate the functioning of this application, thus only rudimentary security rules were enforced by the Firestore and certain additional validation and authorization processing was not implemented on anything other than the client side.

Functional & Non-Functional Requirements

Functional requirements for the app include; safely registering/logging in, having the ability to customize the user profile, creating/joining events, viewing open events, access to real-time weather and map information for events, and having the ability to communicate in-app with other event attendees.

Non-functional requirements for the app include; the ability of the app to process the creation of events quickly, storing created events in a NoSQL database, properly displaying text messaging to maintain chronology, and the displaying of accurate weather and location information.

User Stories

Goal: Enable users to create and manage their identity in the app.

- **Story 1.1**: As a **new user**, I want to **register and create a profile with my name, photo, and a short text summary**, so that I can personalize my account.

- **Story 1.2**: As a **returning user**, I want to **edit my profile information**, so that it reflects my current details when and if they change.

- **Story 1.3**: As a **user**, I want to **set preferences for notifications and visibility**, so that I control what others see and what alerts I receive so as not to be overwhelmed by updates.

Goal: Give users the ability to plan, control, and manage picnic events.

- **Story 2.1**: As a **user**, I want to **create a picnic event with a title, description, park location, date, and time**, so that I can invite others in a simple and straightforward way without relying on other messaging applications or email.

- **Story 2.2**: As an **organizer**, I want to **set whether my event is private (invite-only) or public (open-invite)**, so that I can control attendance to prevent my frenemy from inviting themselves.

- **Story 2.3**: As an **event host**, I want to **edit event details after creation**, so that updates are reflected for attendees and I don't have to message each individual.

- **Story 2.4**: As an **event host**, I want to **cancel an event**, so that attendees are notified and can adjust their plans with minimal inconvenience.

Goal: Help users find and join events that interest them.

- **Story 3.1**: As a **user**, I want to **browse a list of open-invite picnic events near my location**, so that I can join social gatherings and attempt to manage my agoraphobia.

- **Story 3.2**: As a **user**, I want to **filter events by date, park, or group**, so that I can quickly find relevant ones to attend or suggest as an outing to my significant other.

- **Story 3.3**: As a **user**, I want to **RSVP to events** (accept/decline), so that the organizer knows my participation.

- **Story 3.4**: As an arrogant **user**, I want to **view a list of my upcoming events**, so that I can easily check out details to judge which ones are worthy of my presence.

Goal: Enable seamless coordination between participants.

- **Story 4.1**: As an **event participant**, I want to **chat in real-time with others in the event**, so that we can coordinate supplies or meeting spots on the day of the event if the weather allows.

- **Story 4.2**: As an **event host**, I want to **send push announcements to all participants**, so that important updates reach everyone with minimal hassle to myself.

- **Story 4.3**: As a **participant**, I want to **share quick updates or images in the group chat**, so that planning feels collaborative and inclusive.

Goal: Provide users with useful park-related details for planning.

- **Story 5.1**: As a **user**, I want to **see the park location on an interactive map**, so that I know how to get there and not get lost.

- **Story 5.2**: As a **user**, I want to **view basic park amenities (tables, BBQ, restrooms, playgrounds)**, so that I can plan accordingly.

- **Story 5.3**: As a **user**, I want to **see distance and estimated travel time to the park**, so that I can plan my trip by unicycle.

Goal: Keep users informed in real-time.

- **Story 6.1**: As a **user**, I want to **receive a notification when I'm invited to an event**, so that I don't miss out on triggering my bad case of FOMO.

- **Story 6.2**: As a **participant**, I want to **receive reminders before an event starts**, so that I can grocery shop beforehand to bring snacks if necessary.

- **Story 6.3**: As a **participant**, I want to **get instant notifications if the event time, location, or status changes**, so that I always have the latest info when I brag about it to all my friends.

Test Cases

We'd have written them in Dart to test our project, but our knowledge on them is quite minimal. Here is, however, some text cases phrased in pseudo code that generally represents what we otherwise may have written using dart:

Register new user:

```
TEST Register_New_User_Success
  Arrange:
    email = "new@user.com", pass = "StrongP@ss1!"
  Act:
    result = Auth.register(email, pass)
  Assert:
    expect(result.status).toEqual(SUCCESS)
    expect(Profile.get(result.userId)).toExist()
```

Create event:

```
TEST Create_Event_Valid
  Arrange:
    host = loginAs("host@ppp.com")
    payload = {title:"Sunday Picnic", parkId:"MT-123", start:2025-09-15T12:00,
end:2025-09-15T16:00}
  Act:
    evt = Events.create(host.id, payload)
  Assert:
    expect(evt.id).toExist()
```

Discover open-invite events nearby

```
TEST Discover_Open_Invite_Nearby
  Arrange: seedPublicEventsAround(lat,lon, withinKm=5)
  Act:
    results = Discovery.searchNearby(user, {lat,lon}, 5km)
  Assert:
    expect(results.every(e => e.visibility=="PUBLIC")).toBeTrue()
```

RSVP accept increments attendee count:

```
TEST RSVP_Accept_Increments_Count
  Arrange: evt with 0 attendees; user = loginAs("x@ppp.com")
  Act:
    Events.rsvp(evt.id, user.id, ACCEPT)
  Assert:
    expect(Events.get(evt.id).attendeeCount).toEqual(1)
```

Message broadcast to subscribed attendees:

```
TEST Chat_Broadcast
  Arrange: evt with attendees {u1,u2}
  Act:
    Chat.send(evt.id, u1.id, "I'll bring sandwiches")
  Assert:
    expect(Chat.inbox(u2.id, evt.id).last().text).toEqual("I'll bring sandwiches")
```

Roles & Responsibilities

Neither of us had specialized roles in the development of the application, we made efforts to ensure that each of us had exposure and practice using the various technologies that comprised our mobile application. We thus shared and collaborated on the brainstorming of functionality for the program, the creation of mockups, the general program flow, and lastly the data structures that would be committed and persisted to the database. Implementation was also shared, with initial organization for the application being suggested first by Josh and then followed by Kekoa.

In addition to sharing tasks, we also shared work on quality assurance, with each of us reviewing the code the other wrote to ensure that we were comfortable with the functionality. During this process we caught both minor errors, such as misspellings, and more significant ones that might result in dead code or slow execution.

That said, Josh took the lead on the implementation of authentication and the initial integration of the mobile app with Google services and Kekoa undertook work on the chat functionality of the application. While each of us invested a considerable amount of time and energy into the project, we found that given a loss of a team member early on, and not reducing scope a justifiable amount to reflect this change, we were unable to produce a viable minimum product at the end of the project. No animals were hurt during the production of this project, though an animal shaped pillow is now far less fluffy and well-formed.